UNIVERSITI TEKNOLOGI MARA

FACULTY OF MECHANICAL ENGINEERING

Program	**: Bachelor of Engineering (Hons.) Mechanical**
Course	**: APPLIED ELECTRONICS AND MICROPROCESSOR**
COURSE CODE	**: MEC523**
LECTURER'S NAME	**: TS. MUHAMAD FAUZI BIN OTHMAN**
GROUP	**: EMD6M4B**

Mini Project

SMART HYDROPONIC SYSTEM

Bil	STUDENT NAME	STUDENT ID
1.	WAN AMINUDIN BIN WAN AHMAD YUSRI	2019685866

Table of Contents

1.0 Problem Statement 1
2.0 Objectives 1
3.0 Product Description and Details 2
Components used in the system 2
Plan for implementing IoT System to the main structure of Hydroponic System 4
Main Box 4
Interface of application and Webpage 5
4.0 Programming Flowchart 6
5.0 Circuit Diagram 7
6.0 Coding 8
7.0 Result And Discussion 11
ESP32 11
Ultrasonic HC-SR04 for measuring water level 12
pH sensor for measuring pH value of solution 13
Total Dissolved Solid (TDS) sensor to measure 14
8.0 Conclusion 15

1.0 Problem Statement

Hydroponic is difficult to implement since slight changes in the environment have a significant impact on the productivity and quality of farming activity. Utilizing a wireless sensor network to provide an effective decision support system that manages various farm activities and provides relevant farm information. Detailed information about water level, pH level and total dissolve solid. Therefore, the goal of this research is to create an intelligent hydroponic monitoring system that would assist in monitoring the physical conditions of plants.

2.0 Objectives

There are three main objectives in develop smart hydroponic monitoring system using IoT system:

1) To design and implement hydroponics system using IoT for monitoring various parameters such as pH level of water, water level detection, and total dissolve solid

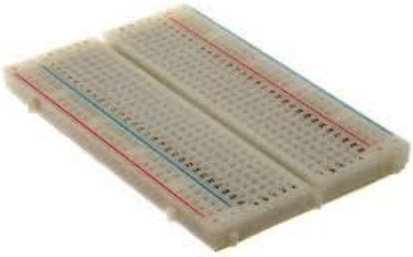
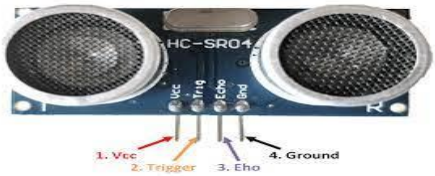
2) To collect required data with sensor through internet connection

3) To provide real time access of the hydroponics system on mobile app to monitor and improve crop yield

3.0 Product Description and Details

Components used in the system:

Table 1: Components used in the system

NO	COMPONENT	FUNCTION
1	ESP32	Allow structuring the program code to run the Smart hydroponic system.
2	Breadboard 	This breadboard is a solderless construction based used for developing an electronic circuit and wiring in connecting the circuit before permanently solder the connection.
3	pH sensor	helps to measure the acidity or alkalinity of the water with a value between 0-14.
4	Ultrasonic sensor 	Used to detect water level of the smart hydroponic system
5	TDS sensor	It is a small hand-held device used to indicate the Total Dissolved Solids in a solution, usually water.

6	LED full Spectrum	Will be place at a strategic place where the light can reach all the plant throughout the system
7	4 channel relay	can be used to control high voltage, high current load such as motor, solenoid valves, lamps, LED, and AC load

Plan for implementing IoT System to the main structure of Hydroponic System

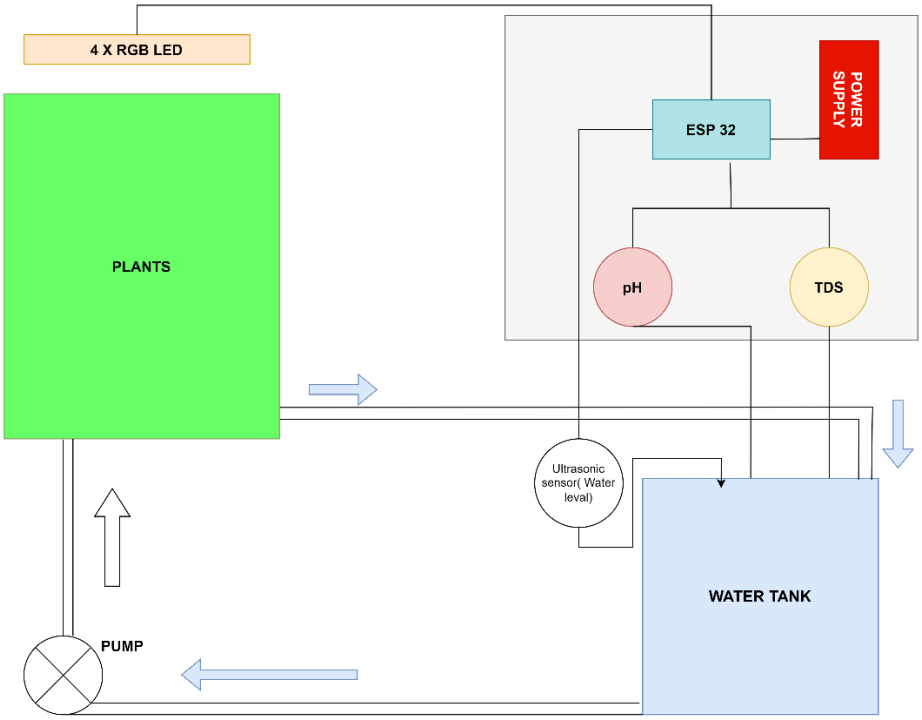


Figure 1: Plan for implementing IoT System to the main structure of Hydroponic System

Main Box

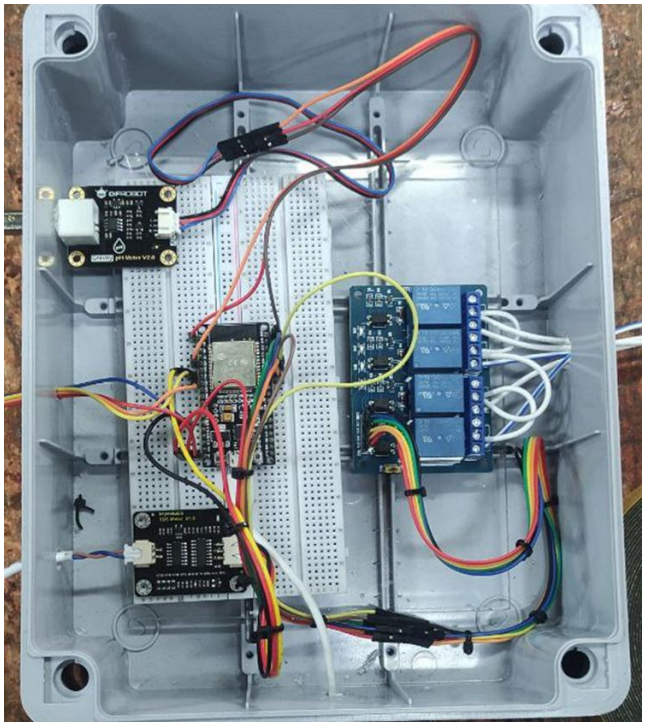


Figure 2: Inside look of the Main Box IoT System consists of ESP32, pH module, TDS module, Relay 4-Channel and breadboard

Interface of application and Webpage

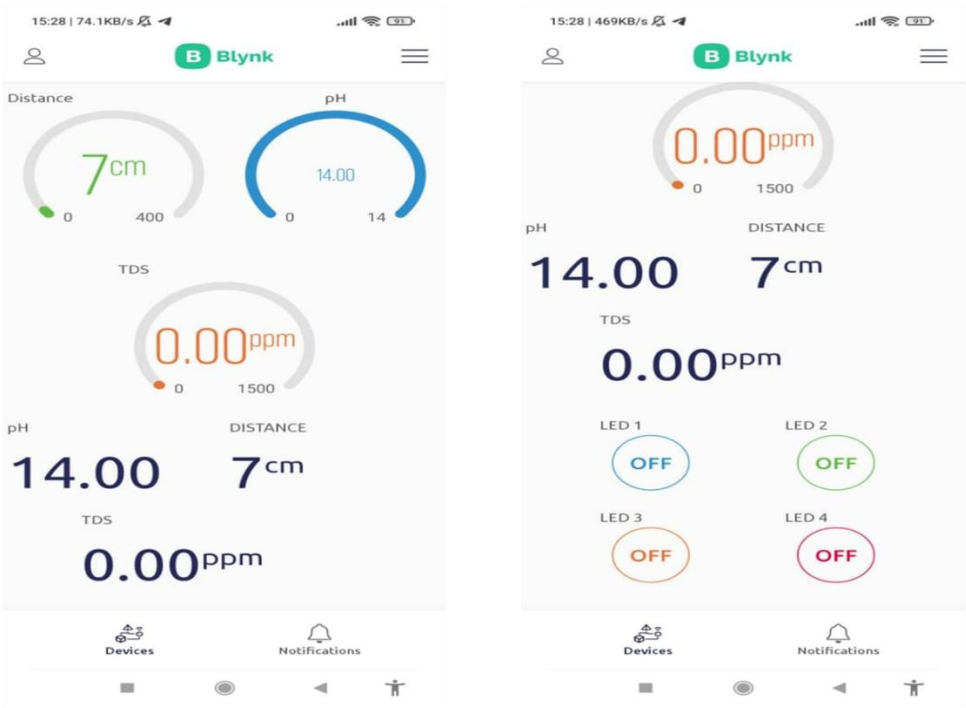


Figure 3: Interface for mobile application

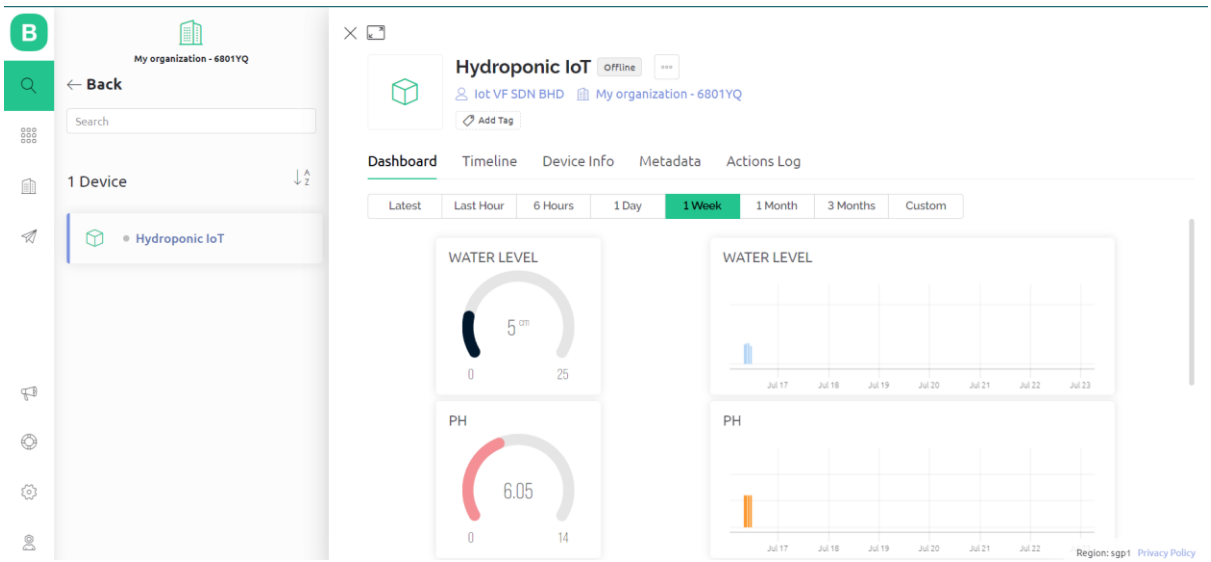


Figure 4: Interface for webpage

4.0 Programming Flowchart

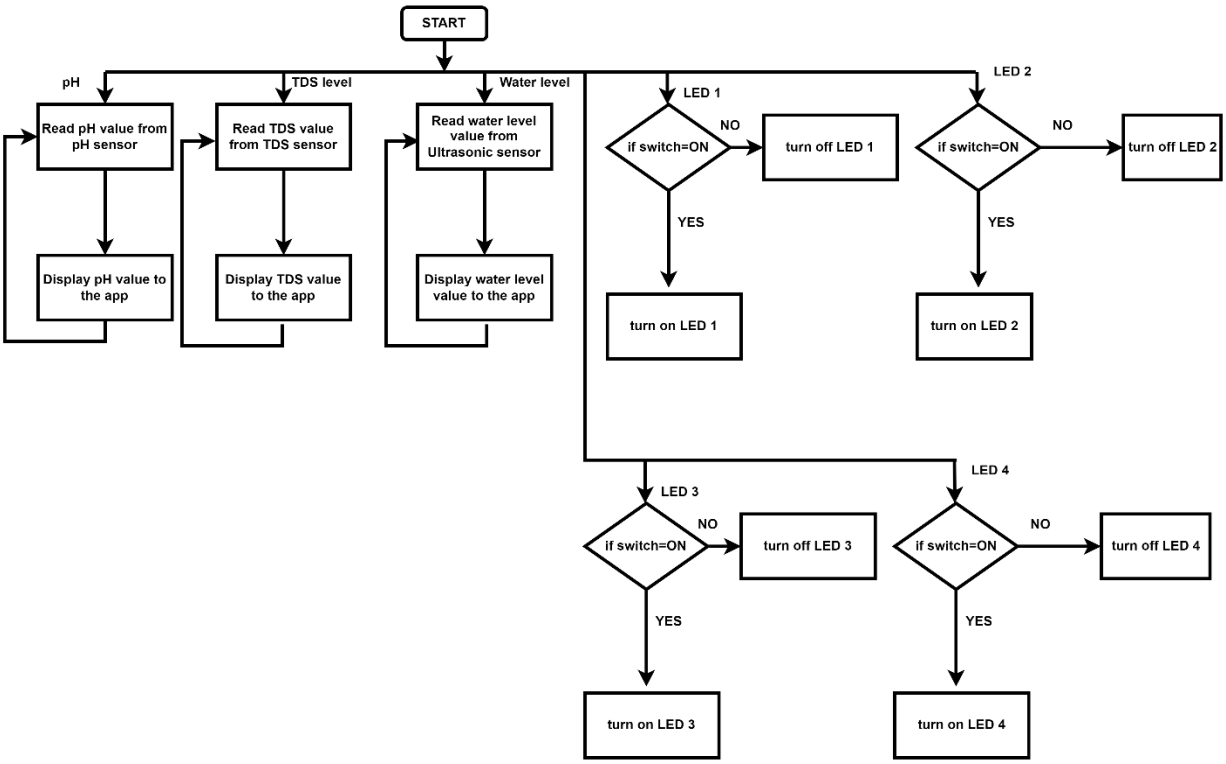


Figure 5: Flowchart of the IoT System

5.0 Circuit Diagram

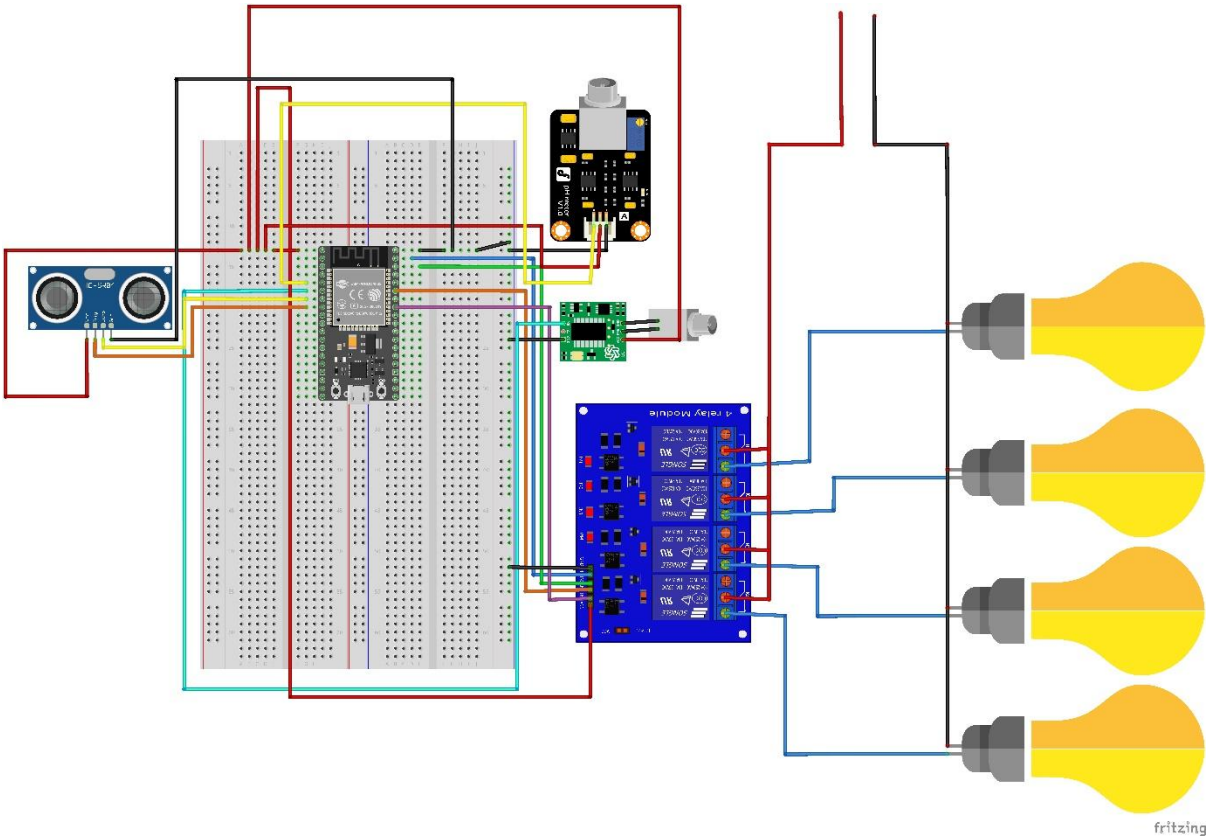


Figure 6: Circuit diagram of the IoT System by FRITZING software

6.0 Coding

```
#define BLYNK_TEMPLATE_ID "TMPLyIoKDvhn"
#define BLYNK_DEVICE_NAME "DIS"

#define BLYNK_FIRMWARE_VERSION "0.1.0"
#define BLYNK_PRINT Serial
#include "BlynkEdgent.h"


//ultrasonic
#define echoPin 32
#define trigPin 33

long duration;
int distance;

long tankDepth= 25;

void ultrasonic()
{
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  duration = pulseIn(echoPin, HIGH);
  distance = duration * 0.034 / 2; //formula to calculate the distance for ultrasonic sensor
  double level = tankDepth- distance;
  Serial.print("Level: ");
  Serial.println(level);
  Blynk.virtualWrite(V0, level);
  delay(500);
}
//ph sensor
#include "DFRobot_ESP_PH.h"
#include "EEPROM.h"

DFRobot_ESP_PH ph;
#define ESPADC 4096.0   //the esp Analog Digital Convertion value
#define ESPVOLTAGE 3300 //the esp voltage supply value
#define PH_PIN 34    //the esp gpio data pin number



void PH_Value()

{
  float voltage, phValue, temperature = 25;
  static unsigned long timepoint = millis();
  if (millis() - timepoint > 1000U) //time interval: 1s
  {
    timepoint = millis();
    //voltage = rawPinValue / esp32ADC * esp32Vin
    voltage = analogRead(PH_PIN) / ESPADC * ESPVOLTAGE; // read the voltage
    Serial.print("voltage:");
    Serial.println(voltage, 4);

    //temperature = readTemperature();  // read your temperature sensor to execute temperature compensation
    Serial.print("temperature:");
    Serial.print(temperature, 1);
    Serial.println("^C");

    phValue = ph.readPH(voltage, temperature); // convert voltage to pH with temperature compensation
    Serial.print("pH:");
    Serial.println(phValue, 4);
    Blynk.virtualWrite(V1, phValue);
  }
  ph.calibration(voltage, temperature); // calibration process by Serail CMD

```

```
}
//TDS sensor
#define TdsSensorPin 35
#define VREF 5.0 // analog reference voltage(Volt) of the ADC
#define SCOUNT 30 // sum of sample point

int analogBuffer[SCOUNT]; // store the analog value in the array, read from ADC
int analogBufferTemp[SCOUNT];
int analogBufferIndex = 0, copyIndex = 0;
float averageVoltage = 0, tdsValue = 0, temperature = 25;


void TDS()
{
  static unsigned long analogSampleTimepoint = millis();
  if (millis() - analogSampleTimepoint > 40U) //every 40 milliseconds,read the analog value from the ADC
  {
    analogSampleTimepoint = millis();
    analogBuffer[analogBufferIndex] = analogRead(TdsSensorPin); //read the analog value and store into the buffer
    analogBufferIndex++;
    if (analogBufferIndex == SCOUNT)
      analogBufferIndex = 0;
  }
  static unsigned long printTimepoint = millis();
  if (millis() - printTimepoint > 800U)
  {
    printTimepoint = millis();
    for (copyIndex = 0; copyIndex < SCOUNT; copyIndex++)
      analogBufferTemp[copyIndex] = analogBuffer[copyIndex];
    averageVoltage = getMedianNum(analogBufferTemp, SCOUNT) * (float)VREF / 1024.0; // read the analog value more stable by the median filtering algorithm, and convert to voltage value
    float compensationCoefficient = 1.0 + 0.02 * (temperature - 25.0); //temperature compensation formula: fFinalResult(25^C) = fFinalResult(current)/(1.0+0.02*(fTP-25.0));
    float compensationVolatge = averageVoltage / compensationCoefficient; //temperature compensation
    tdsValue = (133.42 * compensationVolatge * compensationVolatge * compensationVolatge - 255.86 * compensationVolatge * compensationVolatge + 857.39 * compensationVolatge) * 0.5; //convert voltage value to tds value
    //Serial.print("voltage:");
    //Serial.print(averageVoltage,2);
    //Serial.print("V ");
    Serial.print("TDS Value:");
    Serial.print(tdsValue, 0);
    Serial.println("ppm");
    Blynk.virtualWrite(V2, tdsValue);

  }
}
int getMedianNum(int bArray[], int iFilterLen)
{
  int bTab[iFilterLen];
  for (byte i = 0; i < iFilterLen; i++)
    bTab[i] = bArray[i];
  int i, j, bTemp;
  for (j = 0; j < iFilterLen - 1; j++)
  {
    for (i = 0; i < iFilterLen - j - 1; i++)
    {
      if (bTab[i] > bTab[i + 1])
      {
        bTemp = bTab[i];
        bTab[i] = bTab[i + 1];
        bTab[i + 1] = bTemp;
      }
    }
  }
  if ((iFilterLen & 1) > 0)
    bTemp = bTab[(iFilterLen - 1) / 2];
  else
    bTemp = (bTab[iFilterLen / 2] + bTab[iFilterLen / 2 - 1]) / 2;
  return bTemp;

```

```
}

//led
int RelayPin1 = 23;
int RelayPin2 = 22;
int RelayPin3 = 21;
int RelayPin4 = 19;

BLYNK_WRITE(V3) { //Button Widget is writing to pin V0
  int pinValue = param.asInt();
  digitalWrite(RelayPin1, pinValue);
}

BLYNK_WRITE(V4) { //Button Widget is writing to pin V1
  int pinValue = param.asInt();
  digitalWrite(RelayPin2, pinValue);
}
BLYNK_WRITE(V5) { //Button Widget is writing to pin V2
  int pinValue = param.asInt();
  digitalWrite(RelayPin3, pinValue);
}
BLYNK_WRITE(V6) { //Button Widget is writing to pin V3
  int pinValue = param.asInt();
  digitalWrite(RelayPin4, pinValue);
}



void setup()
{
  Serial.begin(115200);
  EEPROM.begin(32);//needed to permit storage of calibration value in eeprom
  ph.begin();
  //Serial.begin(115200);
  //pinMode(34, INPUT);

  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  pinMode(TdsSensorPin, INPUT);

  //led
  pinMode(RelayPin1, OUTPUT);
  pinMode(RelayPin2, OUTPUT);
  pinMode(RelayPin3, OUTPUT);
  pinMode(RelayPin4, OUTPUT);

  //TURN OFF all Relays on Start
  digitalWrite(RelayPin1, HIGH);
  digitalWrite(RelayPin2, HIGH);
  digitalWrite(RelayPin3, HIGH);
  digitalWrite(RelayPin4, HIGH);


  BlynkEdgent.begin();
  delay(2000);
}

void loop()
{
  BlynkEdgent.run();
  ultrasonic();
  PH_Value();
  TDS();

}

```

7.0 Result And Discussion

ESP32

In this project, microcontroller ESP32 was chosen compared to the Arduino UNO because ESP32 have Wi-Fi feature which is important for IoT. Meanwhile for Arduino UNO, it has no Wi-Fi feature. However extra module Wi-fi can be added to Arduino UNO and it spend a lot of money at the same time, the system is not stable as ESP32 since ESP32 has build-in module Wi-Fi in it. In terms of cost, ESP32 (RM40) is cheaper than Arduino UNO (RM62). Furthermore, ESP32 is compatible with Arduino development environment, Arduino Programing language and Arduino libraries. For development environment, Arduino IDE can be used.

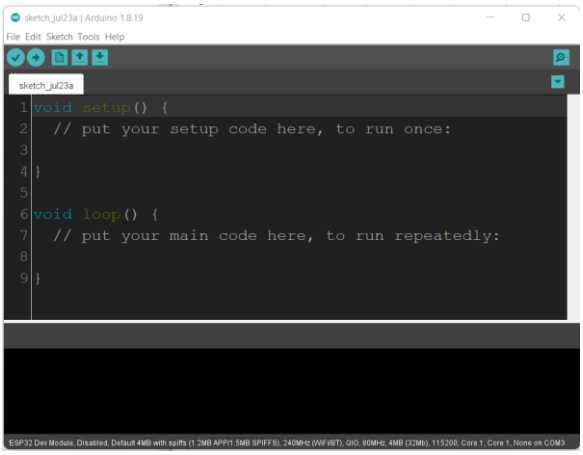


Figure 7: Arduino IDE

It uses a programing language that the most Arduino users have learned for Arduino (C / C++), so no need to learn and adapt new language. Most Arduino libraries can be reuse for ESP32. It can be said that microcontroller ESP32 is very powerful however it is not recommended for the beginners which is not familiar with Arduino.

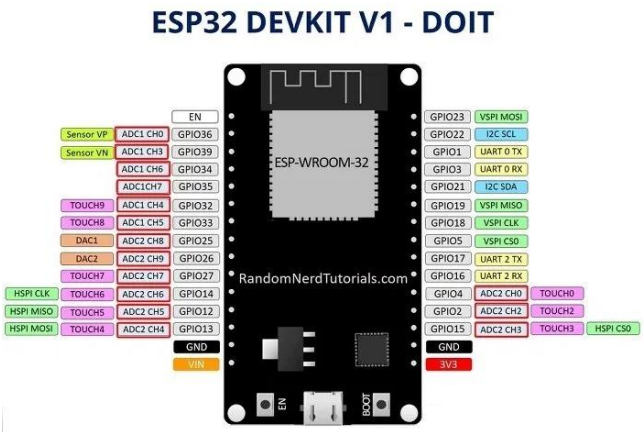


Figure 8: ESP 32 and detail of port

Ultrasonic HC-SR04 for measuring water level

```
1.  //ultrasonic
2.  #define echoPin 32
3.  #define trigPin 33
4.
5.  long duration;
6.  int distance;
7.
8.  long tankDepth= 25;
9.
10. void ultrasonic()
11. {
12.   digitalWrite(trigPin, LOW);
13.   delayMicroseconds(2);
14.   digitalWrite(trigPin, HIGH);
15.   delayMicroseconds(10);
16.   digitalWrite(trigPin, LOW);
17.   duration = pulseIn(echoPin, HIGH);
18.   distance = duration * 0.034 / 2; //formula to calculate the distance for ultrasonic
    sensor
19.   double level = tankDepth- distance;
20.   Serial.print("Level: ");
21.   Serial.println(level);
22.   Blynk.virtualWrite(V0, level);
23.   delay(500);
24.
```

Ultrasonic emitting sound wave at a high frequency to the water surface and time taken for the sound to be reflected are measured to calculate the distance between ultrasonic and water surface. The formula is to calculate distance is shown at coding line 18:

$$distance = \frac{duration * 0.034}{2}$$

After that the water level is calculated as:

$$level = tank\ depth - distance$$

The tank depth value is pre-defined depends to tank used. Figure below show how the water level in the tank

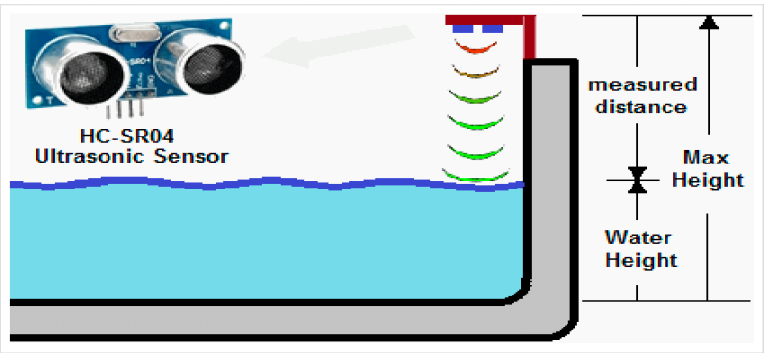


Figure 9: Method to measure water level using ultrasonic sensor

pH sensor for measuring pH value of solution

Figure 10: Buffer solution pH 7.0 and pH 4.0 for calibration of sensor

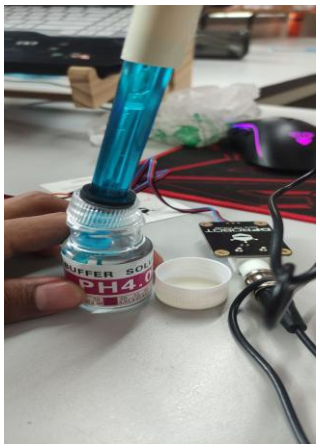


Figure 11: pH probe was immersed in buffer solution to calibrate the sensor

For the first time run, pH Sensor must be calibrated using these 2-buffer solution. pH probe was immersed into these 2- buffer solution. After that the accuracy of pH sensor is checked by pH meter. The accuracy of pH sensor is (+/- 0.01). Figure below show the output from pH sensor. In this project, Lettuce was used as hydroponic plant. The suitable pH for lettuce are 5.5-6.5 which are in weak acidic.

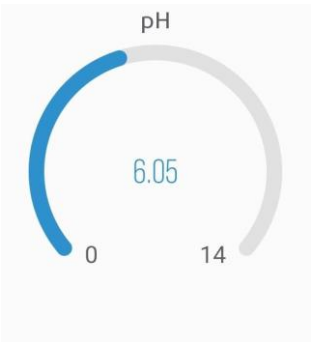


Figure 12: pH reading can be monitor from app

Total Dissolved Solid (TDS) sensor to measure

Total dissolved solid is a measure of the dissolved combined content of all inorganic and organic substances present in a liquid in molecular, ionized, or micro-granular suspended form. TDS concentrations are often reported in parts per million (PPM). Water TDS concentrations can be determined using TDS sensor.

8.0 Conclusion

In conclusion, our project is a system to monitor the parameter in Hydroponic, which is water level, pH of the water, and total dissolved solids. All the parameters were monitored by the specific sensor. The water level was measured by the ultrasonic sensor, the pH sensor will measure the reading of the pH of the water, and the concentration of the total dissolved solid was measured by the TDS sensor. By monitoring all the parameters, the plants will get good quality of supply water and fertilizer which will increase the quality of the crops. The LED light has also been attached to this project to give the plant a healthy amount of light

Overall, the system could be improved by adding more sensors that can control a lot more of the parameters such as light, humidity, and CO2 level. But, due to the lack of financial aid, we could only get fewer sensors that could be used in this project.

The advantages of Portable IoT Hydroponic system are user-friendly because the system can be monitored by using apps named Blynk anywhere and anytime.